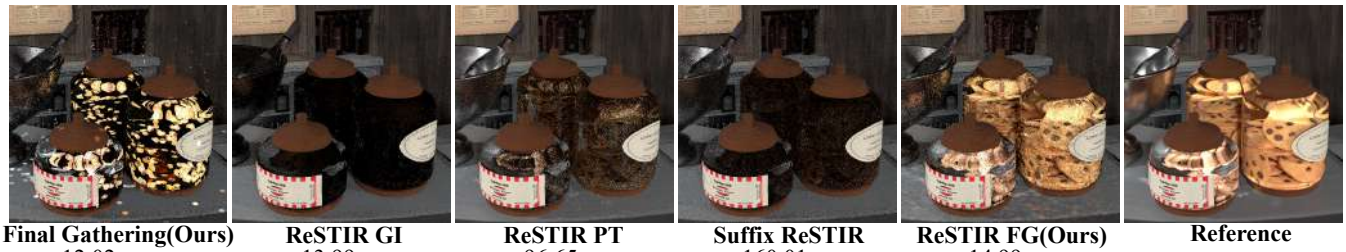
Eurographics Symposium on Rendering (2024)
E. Garces and E. Haines (Editors)

ReSTIR FG: Real-Time Reservoir Resampled Photon Final Gathering

René Kern Felix Brüll Thorsten Grosch

TU Clausthal, Germany



Final Gathering(Ours)	ReSTIR GI	ReSTIR PT	Suffix ReSTIR	ReSTIR FG(Ours)	Reference
12.02 ms	13.99 ms	96.65 ms	160.01 ms	14.99 ms	

Figure 1: *The cookie jar of the bistro rendered using various global illumination resampling techniques at one sample per pixel (SPP). Our approach stands out as the only method capable of approximating the illumination inside of the cookie jar while also achieving a comparable level of quality for global illumination.*

Abstract

*Achieving real-time global illumination for a given scene remains challenging, even with the advent of hardware ray tracing, due to the substantial quantity of rays required. To enhance the quality of the limited number of samples, spatial and temporal resampling can be used. The concept of resampling gained popularity with ReSTIR DI [BWP*20], enabling real-time direct illumination for scenes with millions of lights. This concept was further extended by combining it with path tracing to quickly approximate the indirect illumination (ReSTIR GI [OLK*21]) or correctly approximate global illumination (ReSTIR PT [LKB*22] and Suffix ReSTIR [KLR*23]). However, these algorithms fall short in effectively rendering caustic effects —bundles of reflected or refracted light— often associated with photon mapping.*

We introduce ReSTIR FG, an efficient real time indirect illumination algorithm that combines photon final gathering with the principles of ReSTIR. First, we introduce an efficient photon final gathering scheme, enabling quick consistent offline rendering. Then we combine our photon final gathering with spatiotemporal resampling to allow for real time global illumination. Our algorithm is capable of displaying multi bounce indirect illumination, as well as caustic effects, while remaining competitive in both runtime and quality when compared to the aforementioned state-of-the-art global illumination resampling techniques.

CCS Concepts

• ***Computing methodologies*** → ***Rendering; Ray tracing;***

1. Introduction

Due to the recent advancements in ray tracing hardware, rendering photorealistic images of complex scenes in a short time is more and more possible. However, even with the recent advancements in ray tracing hardware, Monte Carlo based global illumination algorithms such as path tracing [Kaj86], photon mapping [Jen96], or metropolis light transport [VG97], can only produce noisy images due to the low number of samples possible for real time framerates. While high quality denoisers (e.g. SVFG [SKW*17]) can reduce the noise, high quality samples are still needed for good results.

With ReSTIR, Bitterli et al. [BWP*20] introduced an efficient way of rendering millions of lights with only a few shadow rays by reusing a low number of per pixel light samples temporally and spatially. This is achieved by a combination of resampled importance sampling (RIS) [TCE05] and reservoir sampling [Cha82, Vit85]. Reservoir resampling was also used in combination with path tracing to calculate the indirect light by Ouyang et al. [OLK*21] (Re-

© 2024 The Authors.
Proceedings published by Eurographics - The European Association for Computer Graphics.
This is an open access article under the terms of the Creative Commons Attribution License, which permits use, distribution and reproduction in any medium, provided the original work is properly cited.

DOI: 10.2312/sr.20241155

delivered by EUROGRAPHICS DIGITAL LIBRARY www.eg.org diglib.eg.org

STIR GI), Lin et al. [LKB*22] (ReSTIR PT) and Kettunen et al. [KLR*23] (Suffix ReSTIR). However, each of the three algorithms has its limitations, whether struggling with the identification of elusive paths (ReSTIR GI) or demonstrating overall performance that is not sufficient to real-time applications on current hardware (ReSTIR PT and Suffix ReSTIR).

Our work focuses on combining the principles of ReSTIR with a GPU optimized photon mapper to determine the global illumination of a scene in real time. Using photon mapping as opposed to path tracing has the advantage that caustics, bundles of reflected or refracted light, can be found more efficiently. For resampling we use final gathering [Jen96], a variant of photon mapping where path tracing is used on the primary hit, and photon mapping is used for the caustics and the indirect illumination.

Our algorithm demonstrates competitiveness with path tracing based resampling algorithms concerning both runtime and global illumination quality. Notably, as shown in Figure 1, our algorithm is the only one capable of displaying caustic effects in a real time environment. Our contributions are:

- An **optimized GPU based photon final gather algorithm** which effectively renders consistent images with caustics and
- a **resampling scheme for photon final gathering (ReSTIR FG)** based on ReSTIR theory, allowing real time global illumination (Section 4.1), including caustic effects and glass encased objects, that can only be rendered efficiently with both light and camera tracing.
- **Caustic photon collection as a ReSTIR process**, which allows caustics to be visible in real time (Section 4.2).

2. Related Work

Photon mapping [Jen96] is a two-pass approach to solve the rendering equation [Kaj86] via Monte Carlo rendering. In the first pass, light particles called photons are distributed from light sources and stored in a photon map on surface hits. In a second pass, rays are traced from the camera, and the photons are collected within a radius at the first surface hit. Progressive photon mapping [HOJ08] and stochastic progressive photon mapping [HJ09] improve the photon mapping algorithm by progressively shrinking the collection radius depending on the number of photons collected. Knaus and Zwicker [KZ11] showed that the photon collection radius can be reduced globally by a fixed sequence and Kaplanyan and Dachsbacher [KD13] determined the optimal parameters for that sequence. Vertex connection and merging [GKDS12] combines photon mapping and bidirectional path tracing to cover the respective weak points of each technique.

The aforementioned photon mapping algorithms were all designed to run on the CPU. Hachisuka and Jensen [HJ10] introduced stochastic progressive photon mapping that runs on the GPU by utilizing hash grids. Instead of resolving hash collisions, photons stored in the same hash grid cell overwrite each other. Davidovič et al. [DKHS14] noticed that constantly overwriting the last photon leads to bias and proposed stochastically overwriting photons in the same hash cell using reservoir sampling [Cha82, Vit85]. Mara et al. [MLM13] introduced four ways of using photon mapping on the GPU, two using photons as bounded geometry and two based on spatial grids.

The introduction of hardware ray tracing enabled some new ways to efficiently use photon mapping on the GPU. Smal and Aizenshtein [SA19] introduced a photon mapper using reflective shadow maps [DS05], by splatting the photons using rasterization and then filtering to approximate the global illumination in real time. Kim et al. [Kim19] and Yang and Ouyang [YO21] introduced caustic rendering algorithms where the caustics are back projected to the camera. Evangelou et al. [EPVV21] showed a way of using the acceleration structure of the ray tracing API as a photon map. Kern et al. [KBG23] used this to implement a GPU optimized stochastic progressive photon mapper, that also includes an efficient culling algorithm for zero-contribution photons. The photon mapping side of our work is mainly based on the work of Kern et al. [KBG23].

Reservoir Resampling. Bitterli et al. [BWP*20] introduced a very efficient sampling algorithm to render millions of lights with just a couple of light samples per pixel in real time (ReSTIR DI). This is achieved by building upon resampled importance sampling (RIS) [TCE05] and combining it with weighted reservoir sampling (WRS) [Cha82, Vit85]. Wyman and Panteleev [WP21] improved ReSTIR DI speed and memory coherence to make it more suitable for production. Bokšanský et al. [BJW21] introduced a world grid based version of ReSTIR, where ReSTIR is used per grid block to determine a good pool of light samples for each block. This can also be used in next event estimation for better light samples in path tracing [Kaj86]. Tokuyoshi [Tok23] showed an improved version of the biased ReSTIR DI algorithm by rejecting spatial samples based on the shape of the target function.

The ReSTIR algorithm was also used as a base for multiple indirect illumination reservoir resampling algorithms. With ReSTIR GI, Ouyang et al. [OLK*21] introduced a path tracing variant of ReSTIR, where the initial samples are generated with path tracing and then resampled on the first indirect surface hit point. Boissé [Boi21] uses world space reservoirs with a hash grid to efficiently resample NEE samples at the first indirect bounce. Majercik et al. [MMK*22] extends ReSTIR DI to include diffuse global illumination by additionally generating final gather rays, where the diffuse radiance is approximated using DDGI probes [MGNM19, MMSM21] near the final gather location.

With Generalized Resampled Importance Sampling (GRIS), Lin et al. [LKB*22] reformulate the ReSTIR algorithm and introduce a shift mapping algorithm, which allows them to reconnect paths at various hit points. The authors introduce ReSTIR PT as proof of their theory's concept. The GRIS formulation was made more accessible in the SIGGRAPH presentation by Wyman et al. [WKL*23]. Sawhney et al. [SLK*24] showed that correlations in ReSTIR DI and ReSTIR PT can be reduced by interleaving the resampling process with Markov Chain Monte Carlo mutations.

Conditional Resampled Importance Sampling (CRIS) by Kettunen et al. [KLR*23] extends GRIS to conditional probability spaces. Suffix ReSTIR is introduced as the proof-of-concept, where a camera path is split into a prefix and suffix path. The prefix paths end at the second consecutive high-roughness vertex, while the rest of the path is a suffix. Prefixes are only reused temporally with

© 2024 The Authors.
Proceedings published by Eurographics - The European Association for Computer Graphics.

GRIS and the suffixes are reused spatiotemporally using conditional RIS. Additionally, a final gather is performed at the prefix, gathering nearby prefixes and reusing their suffixes.

3. Preliminaries

The rendering equation [Kaj86] is the basis for lighting calculations in computer graphics: The outgoing radiance L at a given point x with the outgoing direction $\vec{\omega}_o$ can be determined as follows:

$$L(x,\vec{\omega}_o) = L_e(x,\vec{\omega}_o) + \int_\Omega \rho(x,\vec{\omega}_i,\vec{\omega}_o) L_i(x,\vec{\omega}_i) \cos\theta \, d\vec{\omega}_i \tag{1}$$

Where L_e is the emitted radiance at the point x, Ω is the integration domain, ρ the BSDF, L_i the incoming radiance with the incoming direction ω_i and $\cos\theta$ is the cosine of the angle between the incoming direction and the surface normal. For simplicity, we define the term in the integral as:

$$I = \int_\Omega f(s) ds \tag{2}$$

with s being a sample consisting of $x, \vec{\omega}_o$ and $\vec{\omega}_i$.

3.1. Photon Mapping

The outgoing radiance for a given point x can be approximated with photon mapping [Jen96]. Before calculating the outgoing radiance, the photons first need to be distributed via light tracing from the scene's light sources. On each surface hit, a photon p, containing the flux $\Delta\phi_p$ and the incoming direction $\vec{\omega}_p$ is stored. In a second pass, rays are distributed from the camera and on the first diffuse surface hit x the outgoing radiance can be approximated as:

$$I \approx \langle I \rangle_{PM} = \frac{1}{\pi r^2} \sum_{p=1}^{M} \rho(x,\vec{\omega}_p,\vec{\omega}_o) \, \Delta\phi_p \tag{3}$$

where M is the number of photons inside in the radius r.

To efficiently collect the photons, a collection data structure is needed. As shown by recent works [EPVV21, KBG23] the ray tracing API's acceleration structure is the fastest option to collect photons on the GPU. The photons are collected via inverse radius search [EPVV21], where the search radius is mapped to each photon instead of a radius search query at the first surface hit point. Each photon is stored as a sphere in the ray tracing API's acceleration structure. As spheres are not supported by default, axis aligned bounding boxes (AABB) with sphere intersections shaders are used instead. Photons can then be collected by distributing an infinitely small ray at the first surface hit position x. The anyHit shader of the ray tracing API will be called for every photon the ray intersects with. Kern et al. [KBG23] additionally introduced a hash grid based culling algorithm for non-contributing photons, that is especially useful for bigger scenes, where many photons are outside the camera view, as well as a stochastic collection scheme based on reservoir sampling [Vit85] to reduce collection times at high photon density areas.

A single photon map can be divided into two types of photon maps, a caustic photon map that contains photons that were previously reflected or refracted by a highly specular surface and a global photon map that contains the rest of the photons. The caustic photon map often uses a smaller collection radius than the global photon map. Photon mapping can be combined with final gathering [Jen96], where the global photon map is evaluated at the first diffuse indirection. The collection position for global photon map is determined by distributing a ray from the first diffuse surface hit. The caustic photon map and the direct light is evaluated directly on the first diffuse surface hit.

3.2. ReSTIR

In this section we explain the basics of ReSTIR needed for our algorithm using the theory introduced by GRIS [LKB*22]. We follow the simpler notation used in "A gentle introduction to ReSTIR" [WKL*23] and highly recommend it for a more in-depth explanation of the theory behind ReSTIR.

3.2.1. RIS

The integral I from Equation 2 can be approximated via Monte Carlo integration using M randomly selected samples $X_1,...,X_M$ and can be written in the following form:

$$I \approx \langle I \rangle = \sum_{i=1}^{M} \frac{1}{M} \frac{f(X_i)}{p_i(X_i)} \tag{4}$$

where the contribution of each random sample X_i is divided by its probability density function p_i and gives an unbiased estimate of the integral, as long as $p(s) > 0$ where $f(s) \neq 0$. Note that we use capital letters X to refer to random samples, in contrast to the defined s in Equation 2.

The closer the PDF $p(s)$ is to $f(s)$ the better is the Monte Carlo estimate for the sample s. However, due to the complexity of $f(s)$ it is often impossible to directly choose a $p(s)$ that is proportional to all parts of $f(s)$. Talbot et al. [TCE05] showed with resampled importance sampling (RIS) that multiple samples generated from a simple (non-optimal) source distribution p can be used to generate a sample proportional to $f(s)$ by weighting the generated samples with a source distribution p and a target function $\hat{p}$ (e.g. $\hat{p} \propto f(s)$). From M generated candidates, a random candidate with index i can be chosen with the discrete probability:

$$Pr(\text{choose } i) = \frac{w_i}{\sum_{j=1}^{M} w_j} \quad \text{with } w_i = \frac{1}{M} \frac{\hat{p}(X_i)}{p(X_i)} \tag{5}$$

The 1-sample RIS estimator with the chosen candidate sample $Y \equiv X_i$ can be defined as follows:

$$\langle I \rangle_{RIS} = \frac{f(Y)}{\hat{p}(Y)} \cdot \left(\sum_{j=1}^{M} \frac{1}{M} \frac{\hat{p}(X_j)}{p(X_j)} \right) \tag{6}$$

In addition to RIS, unbiased contribution weights W_X are necessary. Given X, their expected values can be treated as $1/p(X)$. With the generalized formulation of RIS [LKB*22] we can use:

$$W_X = \frac{1}{\hat{p}(X)} \cdot \sum_{j=1}^{M} w_j \tag{7}$$

$$\langle I \rangle_{RIS} = f(Y) \cdot W_Y \tag{8}$$

and if the PDF $p(X_i)$ to W_{X_i} is known, $W_{X_i} = \frac{1}{p(X_i)}$ can be used

© 2024 The Authors.
Proceedings published by Eurographics - The European Association for Computer Graphics.

instead. Additionally, W_X can be used to reproduce the resampling weights w_i from Equation 5:

$$w_i = m_i(X_i)\hat{p}(X_i)W_{X_i} \tag{9}$$

with $m_i(s)$ being MIS weights ($1/M$ if all X_i are equally distributed). The following section will go over the importance of the MIS weights.

3.2.2. MIS

The factor $1/M$ from Equation 5 only holds if all samples X_i are equally distributed and are valid for all possible target functions $\hat{p}$. If this is not the case, it will result in a biased W_X. As soon as neighboring samples are introduced, this constraint cannot be guaranteed anymore (for a good visualization see [WP21, Section 4]). This problem can be fixed with MIS. If the input PDFs of the samples are known, the MIS balance heuristic [VG95] can be used to replace $1/M$, eliminating the bias:

$$m_i(s) = \frac{p_i(s)}{\sum_{j=1}^{M} p_j(s)} \tag{10}$$

3.2.3. Reservoir Resampling

Reservoir resampling allows for GPU friendly chaining of RIS by utilizing weighted reservoir sampling (WRS) [Cha82, Vit85]. Instead of needing to store all candidates upfront before selecting an output, WRS allows for sampling in a streaming fashion by maintaining a small reservoir. An example of a reservoir is shown in Algorithm 1, where the reservoir consists of a chosen output sample Y, the unbiased contribution weight W_Y, the sum of the weights w_{sum} and a confidence c that is explored in the next section.

3.2.4. Confidence Weights

Without any confidence weights, resampling will assign equal weights when reusing. This can be sub-optimal, as a new reservoir would have the same weight as past reservoirs containing the history of multiple frames, effectively losing roughly 50% of accumulated history. The confidence weights can be included into MIS (Equation 10) to fix this problem:

$$m_i(s) = \frac{c_i p_i(s)}{\sum_{j=1}^{M} c_j p_j(s)} \tag{11}$$

Each new reservoir is assigned a confidence of $c = 1$. When combining reservoirs, the confidence of both reservoirs is added, simulating an effective sample count. The GRIS paper [LKB*22] showed that limiting the confidence weight is essential for avoiding correlations in ReSTIR.

3.2.5. Shift Mappings

Shift mappings T are essential for path reuse, as it allows shifting a sample X_i from the arbitrary domain Ω_i into the generic domain Ω. If a shift mapping maps a path between domains ($Y = T(X)$), then the path densities need to be modified with Jacobian determinants $|T'(x)|$. With shift mapping, the resampling weights (Equation 5) are defined as follows:

$$w_i = m_i(Y_i)\hat{p}(Y_i)W_{X_i}|T'(X_i)| \tag{12}$$

Algorithm 1: RIS between Domains [WKL*23]

```
1  class Reservoir
2      Y, W_Y ← ∅, 0 //Output Sample
3      w_sum ← 0 //Sum of weights
4      c ← 0 //Confidence weight
5      function update (X_i, w_i, c_i)
6          w_sum ← w_sum + w_i
7          c ← c + c_i
8          if rand() < (w_i / w_sum) then
9              Y ← X_i
10 function Resample (M)
11     Reservoir r
12     for i ← 1 to M do
13         Generate X_i //Can also be taken from a reservoir pool
14         Y_i ← T_i(X_i) //Shift X_i into Ω
15         if Y_i ≠ ∅ then
16             w_i ← m_i(Y_i) p̂(Y_i) W_{X_i} |T_i'(X_i)|
17         else
18             w_i ← 0 //If Shift fails
19         r.update(Y_i, w_i, c_i)
20     end
21     if r.Y ≠ ∅ then
22         r.W_Y ← 1/p̂(r.Y) r.w_sum
23     r.c ← min(r.c, c_cap)
24     return r
```

Additionally, the MIS weight needs to be adjusted when using a shift mapping. Lin et al. [LKB*22] uses the target function as a proxy for p:

$$\hat{p}_{\leftarrow i}(y) = \begin{cases} \hat{p}_i(T_i^{-1}(y)|T_i^{-1'}(y)|), & \text{if } y \in T_i(\text{supp } X_i) \\ 0, & \text{otherwise} \end{cases} \tag{13}$$

meaning that the sample y evaluates the target function $\hat{p}_i$ at the sample location of X_i in the domain Ω_i, when y could be created in domain Ω_i. With this the MIS weight is:

$$m_i(s) = \frac{c_i \hat{p}_{\leftarrow i}(s)}{\sum_{j=1}^{M} c_j \hat{p}_{\leftarrow j}(s)} \tag{14}$$

Algorithm 1 (Line 10-24) shows the process of resampling M reservoirs. The sample generation (Line 13) is interchangeable with fetching a reservoir from a past or neighboring pixel.

4. Reservoir Resampling with Photon Final Gathering

Combining final gather photon mapping with ReSTIR [BWP*20, LKB*22, WKL*23] allows for an indirect illumination algorithm that can render high-quality images in real-time by resampling photon final gather samples spatially and temporally, to increase the effective sample count drastically (**ReSTIR FG**). For direct light we opted to use ReSTIR DI [BWP*20].

For our ReSTIR FG, photons are distributed every frame and stored in a global or caustic photon map. Similar to photon final gathering, the global photon map will be used to approximate the

© 2024 The Authors.
Proceedings published by Eurographics - The European Association for Computer Graphics.

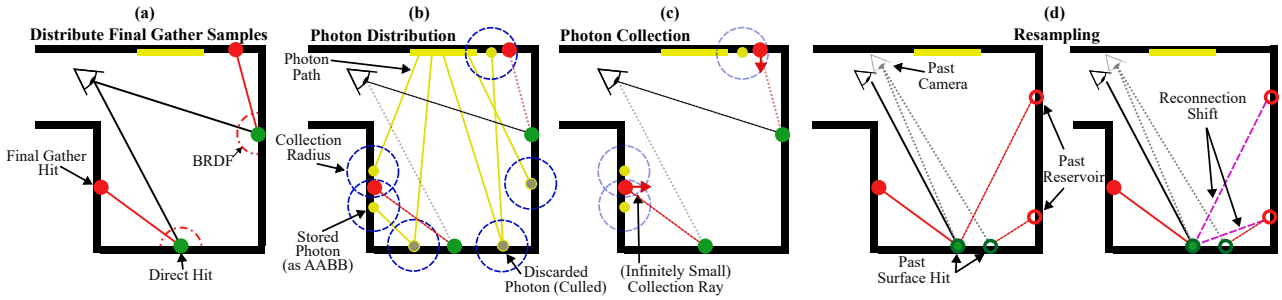


Figure 2: *Calculation of indirect light via photon final gathering and ReSTIR (excluding caustics).* ***(a)*** *An initial photon final gather sample ● is generated from the first diffuse hit (here direct hit) ●.* ***(b)*** *Photons are distributed from the light and traced through the scene. On a surface hit, a photon is stored ● if it is in proximity of the final gather sample ●. The photon is discarded ● if it is not in the proximity of the final gather sample ● (Photon Culling [KBG23]).* ***(c)*** *The photon acceleration structure is built and the photons are collected with an infinitely small ray → from the final gather samples ●.* ***(d)*** *Photon final gather samples from past iterations ○ are reused via ReSTIR. Reconnection shift [LKL*13] — is used to shift the samples from past ● or neighboring domain ○ into the current domain, allowing reuse.*

radiance for a random direction, while the caustic photons are collected directly. Section 4.1 shows how we generate the photon final gather samples and resample them to estimate the global illumination without caustics. In Section 4.2 we use ReSTIR to collect the photons, replacing the radiance estimate (Equation 3). Because the first camera surface hit is not ideal for photon final gathering, we describe how tracing to the first surface is handled in Section 4.3. Additionally, this sections clarifies the definition of a diffuse surface. Section 4.4 describes various optimizations to increase performance.

4.1. Resampling with Photon Final Gather Samples

A simplified overview of the photon final gather part of ReSTIR FG looks as follows, with a visual representation shown in Figure 2:

1. Trace from camera to the first diffuse surface hit. (Section 4.3)
2. Generate a random photon final gather sample by tracing a random ray according to the BRDF of the surface (Figure 2a).
3. Distribute photons from the light source and store photons on surface hits (Figure 2b).
4. Build photon acceleration structure. Collect photons with an infinitely small ray [EPVV21, KBG23] to approximate the radiance of the final gather sample (Figure 2c).
5. Resample the photon final gather samples using ReSTIR (Algorithm 1) with reconnection shift [LKL*13] (Figure 2d)

We now go over the process in more detail. The initial photon final gather sample generation scheme is built with photon culling [KBG23] in mind to reduce the acceleration structure build time and the photon storage memory requirements on bigger scenes. Therefore, the initial final gather collection samples need to be generated prior to distributing the photons to generate the photon culling hash grid (shown in Figure 2b). Additionally, during step 1, the first diffuse hit is also marked in the photon culling hash map.

Generate Photon Final Gather Sample: The initial photon final gather samples are generated using a ray starting from the first (diffuse) camera surface hit which is traced in a random direction determined by BRDF sampling (Figure 2a). When a surface is hit, the sample Y in the reservoir is initialized with the surface information and an empty radiance, which will be obtained at a later step. The unbiased contribution weight W_Y is initialized with the PDF of the photon final gather sample $(W_Y = \frac{1}{p(x)})$.

Photon Generation: For photon generation, a predetermined number of photons are traced from the light sources with light tracing (Figure 2b). When a surface is hit, the photon is either stored in the global or caustic photon map, depending on if the previous bounce was specular or diffuse. The photon is only stored if the bit in the photon culling hash map is set. The photon index is determined by an atomic counter, ensuring that each photon index is used once. The photon information is stored in two buffers per photon map, an AABB buffer for the acceleration structure and a data buffer(incoming direction, flux), both with the same index. After tracing the photons, the acceleration structure for each photon map is built using the ray tracing API.

Photon Collection: The photon collection is used to approximate the radiance of the photon final gather sample $(r.Y)$ that was previously generated in Figure 2a. This process is shown in Figure 2c. We opted to use the reservoir sampling based collection approach described by Kern et al. [KBG23]. This method involves using an infinitely small ray with a custom sphere intersection shader to collect photons in the anyHit shader. Reservoir sampling [Vit85] is applied to limit the evaluation of the radiance estimate (Equation 3) to a few photons (e.g. 3). As demonstrated by Kern et al., the reservoir-based collection scheme proves advantageous in situations where multiple high-density photon areas exist, a common occurrence with a fixed radii.

After approximating the radiance of the photon final gather sample, the reservoir can be used for resampling. When resampling is not used, the steps shown in Figure 2a-c can be used for a consistent GPU final gathering photon mapper.

The caustic photons are also collected in this step with a similar method, which is further explored in the Section 4.2,

Resampling: The effective sample count can be increased drastically be reusing photon final gather samples that were generated

© 2024 The Authors.
Proceedings published by Eurographics - The European Association for Computer Graphics.

Algorithm 2: ReSTIR FG

```
function ReSTIR_FG (uint2 pixel, Surface directSf)
    Reservoir fgR ← {} //Work reservoir (FG)
    Reservoir causticR ← {} //Work reservoir (caustics)
    //Trace to first (diffuse) surface (can be directSf)
    Surface S ← traceToFirstDiffuse(directSf)
    //Trace the final gather ray and initialize the reservoir.
    fgR.Y, fgR.W_Y ← traceFinalGather(S)
    //Distribute the photons and build acceleration structure
    AccelerationStructure photonAS
    photonAS ← distributePhotonsAndBuildAS()
    //Approximate radiance for the final gather reservoir
    fgR ← collectPhotons(photonAS, fgR)
    //Generate and initialize caustic reservoirs at position S
    causticR ← collectPhotons(photonAS, causticR)
    //Spatiotemporal resampling (Algorithm 1)
    fgR ← resample(pixel, fgR, S)
    causticR ← resample(pixel, causticR, S)
    //Store reservoir and surface for next iteration
    setPastData(pixel, fgR, causticR, S)
    //Evaluate the (resampled) reservoir (Equation 8)
    indirectRadiance ← evalReservoir(fgR, S)
    causticRadiance ← evalReservoir(causticR, S)
    return indirectRadiance + causticRadiance
```

in previous iterations or at neighboring pixels (Figure 2d). The basic theory behind the ReSTIR algorithm was explored in Section 3.2 and is shown in Algorithm 1.

For resampling, the reservoir from the previous iteration is used for the temporal and spatial resampling step, eliminating the need for two passes with a barrier [WP21]. The only downside is, that spatial resampling is delayed by one frame. The temporal resampling is performed at the reprojected pixel from last frame. For spatial resampling, a random neighboring pixel is used. While arbitrary numbers of resampled spatial samples are possible we typically only use one spatial sample, increasing it to two spatial samples when the temporal sample is invalid. A higher number of spatial samples have diminishing returns, as Wyman and Panteleev showed [WP21].

After fetching the spatiotemporal sample, we compare the surfaces that generated the reservoir samples against each other (relative depth, normal) and ignore the reservoir if the surfaces differ too much. Then a reconnection shift [LKL*13] is performed using the solid angle parametrization of the Jacobian determinant [KMA*15]. If the shift mapping fails, the reservoir weight is set to zero ($w_i = 0$). The resampling is first performed without evaluating the MIS weights ($m_i = 1$) to select a sample. The correct reservoir weight is then calculated in a second loop where the MIS weight (Equation 14) of the chosen sample is evaluated with all used spatiotemporal samples. Note that a proper evaluation of the shift mapping MIS weights involves tracing a visibility ray per sample, checking if the sample could have been generated from the domain of the chosen sample. To increase performance, this visibility check can be omitted which can introduce a slight bias.

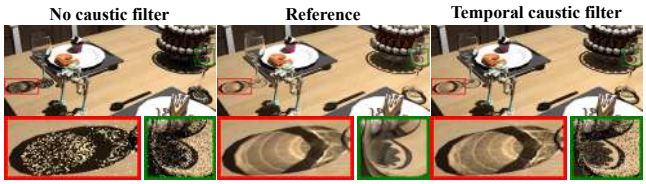


Figure 3: *1SPP caustics (left) and caustics collected via ReSTIR (right). With collection via ReSTIR the quality is closer to the reference (middle). For sharper caustics, the Kitchen scene from this figure uses a single point light instead of area lights.*

After resampling, if the final sample is valid, the unbiased contribution weight $r.W_Y$ is calculated. The reservoir can be used for shading (Equation 8), after checking the visibility of the final sample.

4.2. Collecting Caustic Photons with ReSTIR

Tracing the number of photons necessary to get sharp caustics in real time (one iteration) is not feasible, as we would need to distribute millions or even billions of photons each frame. We opted to also use ReSTIR to reuse the radiance estimate (Equation 3), increasing the effective sample count.

For this, we transform the radiance estimate into a form that is able to perform RIS:

$$\langle I \rangle_{PM} = \underbrace{\frac{M}{\pi r^2}}_{\frac{1}{p(X_p)}} \sum_{p=1}^{M} \underbrace{\frac{1}{M} \rho(x, \vec{\omega}_p, \vec{\omega}_o)\, \Delta\phi_p}_{\frac{1}{M}\hat{p}(X_p)} \tag{15}$$

One photon is selected via RIS to represent the radiance estimate. Because the factor M cancels itself out, each photon can be added to the reservoir with the resampling weight $w_i = \frac{\hat{p}(X_p)}{\pi r^2}$. After collecting the photons with Equation 15, the weight sum needs to be divided by the target function of the selected photon to initialize the unbiased contribution weight (Equation 7).

With the photon reservoir created, resampling via ReSTIR can now be performed (Algorithm 1). Each reservoir candidate needs to be shifted to the current domain. This is done by checking if the distance of the selected photon sample in the reservoir is smaller or equal to the collection radius. Additionally, the MIS balance heuristic is vital (Equation 14) to ensure that no additional bias appears.

While the resampling process in itself is not biased, photon mapping with a fixed radius is. For offline rendering, SPPM can be used to reduce the collection radius every frame for consistent rendering.

We generate the caustic reservoir in the photon collection step (Figure 2c) by collecting the caustic photons at the first diffuse camera surface. The resampling pass is performed separately from the photon final gather resampling pass. The full ReSTIR FG algorithm can be seen in Algorithm 2.

4.3. First diffuse Surface

The first camera surface hit can be acquired using a G-Buffer. However, the surface of the first camera hit is not always diffuse. Sam-

© 2024 The Authors.
Proceedings published by Eurographics - The European Association for Computer Graphics.

pling photon final gather rays from specular materials can lead to problems as illustrated in Figure 4 (Top left). Continuing the path on specular materials works well for delta reflections or refractions (e.g. glass, mirror), as the reflected path is well-defined. However, this can be disadvantageous on rough metals as the direct light would depend on random paths hitting the light source (Figure 4 Top right). On those rough metals we opted to use a hybrid system, where ReSTIR DI is evaluated directly on the rough metals, while our ReSTIR FG is evaluated on the first diffuse hit. However, this causes the direct light reflected by the metal at the first diffuse hit to be missing. For the example shown in Figure 4, this would result in the counter, reflected in the metal bowl, appearing darker because the direct light on the counter would be missing. In these cases, we collect photons with a path length of zero on the first diffuse hit (Figure 4 Bottom left). In our implementation, photons that were not previously reflected are marked with a flag, so they can be differentiated from the other photons in the global photon map. Similar to caustic photons, resampling can be applied to these photons (Section 4.2).

Materials are often not purely diffuse and contain a specular part. However, as long as the BRDF lobes are not too thin, resampling is preferred over tracing the path further. Therefore, we chose to use a hard cutoff based on material properties (roughness & diffuse probability) to determine which surfaces are diffuse. Additionally, this is advantageous for resampling, as nearby pixels stop on nearby surfaces.

4.4. Optimizations

To improve the relative quantity of caustic photons, global photons are stored with a fixed probability (e.g. 30%) and the flux of these global photons is increased with the inverse of this probability (e.g., $\frac{1}{0.3}$). While this increases photon tracing times, it drastically improves caustics without noticeably affecting the final gather quality. Additionally, when too many photons overlap in a small area, it causes significant performance drops during acceleration structure building and photon collection due to the sequential nature of the anyHit shader. Therefore, storing fewer but brighter photons is preferred. Russian Roulette can also be used to terminate low-contribution paths early.

Even with a reduced probability of storing global photons, we observed significant overlapping of photons near the surfaces of emitters (e.g., lampshades). To mitigate the negative performance impact, the photon storing probability can be reduced until the photon traveled a certain distance. The photon flux is then increased by the inverse of this reduced probability. We set the threshold experimentally to 1% of the scene's extent. We did not notice any seams with a hard cutoff, but gradually increasing the probability is also an option.

Due to acceleration structure building, the photon buffer size needs to be estimated before distributing the photons. Kern et al. [KBG23] observed that the number of stored photons remains similar when the camera is stationary. Therefore, we used a fixed photon buffer size and dynamically adjusted the number of dispatched photons based on the number of photons stored in the last frame. This can lead to a short timed bias on sudden, strong movements to a region where more photons are stored.

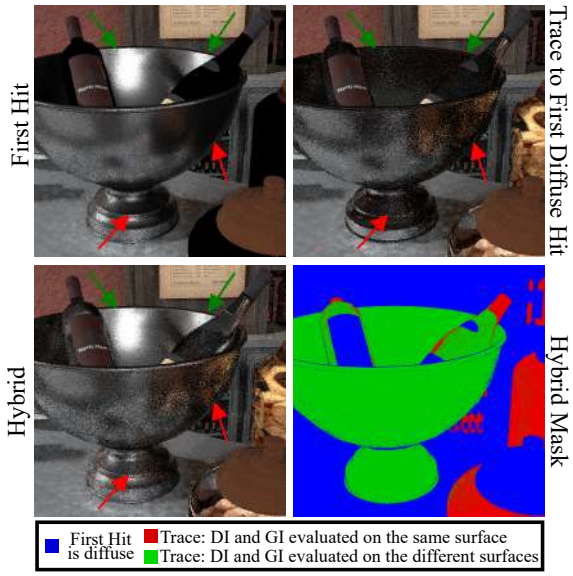


Figure 4: *Different methods to evaluate ReSTIR DI (DI) and ReSTIR FG (FG). Top left: Highly specular materials cannot connect to reservoirs due to the infinitely thin BSDF (DI & FG). For rough metals, DI works great but recursion is missing in FG ⟶. Top right: When tracing to the first diffuse hit, we lose the advantage of ReSTIR for hits on rough metals ⟶. Emissive light is still found with camera path tracing, however, due to the metal BSDF it is very noisy. Bottom: By using a hybrid system we can resolve both problems. Highly specular materials (e.g. glass, mirrors) are traced for both ■. For DI we stop on rough metals, while we trace further for FG until the first diffuse surface hit ■. As DI and FG are evaluated on different surfaces in the ■ case, the direct light for the FG surface is missing. The missing direct light is included by collecting direct photons from the global photon map.*

5. Implementation

We implemented our algorithm in the Falcor framework [KCK*22] with full source code available (see Supplemental Materials). For the comparisons, all algorithms use ReSTIR DI (RTXDI) [BWP*20, WP21] for direct light and a ray traced V-Buffer to obtain primary hit information. For ReSTIR PT [LKB*22] and Suffix ReSTIR [KLR*23] the reference implementation provided by the authors was used. It is important to note that these implementations are proof-of-concept and have not been optimized. For Suffix ReSTIR, the default parameter provided by the authors was used (4 integration prefixes, 1 final gather suffix). ReSTIR GI [OLK*21] was re-implemented by us and uses a similar surface handling as ReSTIR FG (Section 4.3), but always traces to the first diffuse hit. For all comparisons, a path length of 10 was used.

We added support for the NVIDIA real time denoiser (NRD) 3.1 and deep learning super sampling (DLSS) 3.1 in our implementation of ReSTIR FG and GI, which is compared in Section 6.5. Unfortunately, the implementations of ReSTIR PT and Suffix ReSTIR provided by the authors do not include any denoisers. For the results, we used two versions of ReSTIR FG with different photon

© 2024 The Authors.
Proceedings published by Eurographics - The European Association for Computer Graphics.

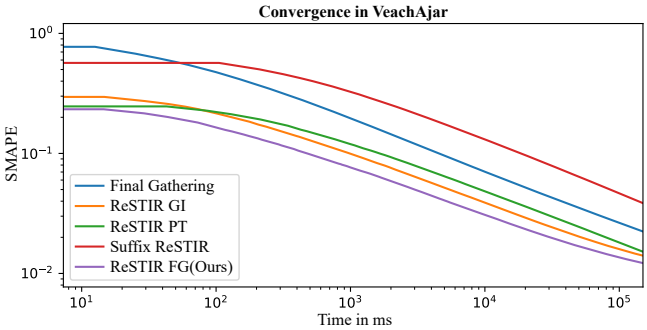


Figure 5: *Convergence graph in VeachAjar. The photon radius for final gathering and ReSTIR FG was reduced every frame (SPPM).*

buffer sizes. A quality version with a global photon buffer size of 400.000, which results, depending on the scene, in 1.4M to 0.8M photons distributed and a performance version with 1/4 the photon buffer size (100.000). The performance version is used for the comparisons in Figure 7, Figure 9 and Table 1, while the quality version is used in all other figures.

	Bistro		Kitchen	
Part	Quality	Perf.	Quality	Perf.
Trace First Diff.	1.26		1.29	
Init. FG Sample	1.47		1.14	
Photon Gen.	2.40	0.68	1.59	0.46
Build AS	1.82	0.84	1.90	0.87
Collect Photons	3.86	2.88	11.74	4.72
Resampling	1.53		1.42	
Caustic Resampling	0.84		0.92	
Eval. Reservoir	0.92		0.87	
ReSTIR FG Total	14.10	10.42	20.70	11.49
VBuffer	0.67		0.57	
RTXDI	4.92		5.35	
NRD	4.02		4.00	
DLSS	1.39		1.35	
Misc.	0.72		0.76	
Total + Denoising	25.82	22.14	32.73	23.52

Table 1: *Render times in ms for each part of our ReSTIR FG in two different quality settings for two scenes. Additionally, times for VBuffer, RTXDI, miscellaneous and denoising are also shown. Miscellaneous includes the tone mapper and denoising extras.*

6. Results

ReSTIR-FG is evaluated and compared to other algorithms on a range of test scenes. All render times are measured on an NVIDIA RTX 3070Ti using a rendering resolution of 1920×1080. The references were rendered with GPU photon final gathering and stochastic progressive photon mapping using the radius reduction by Knaus and Zwicker [KZ11] ($\alpha = 2/3$, 100.000 SPP).

To compare the images, we use SMAPE: $\frac{1}{n}\sum_{t=1}^{n}\frac{|F_t - A_t|}{(|A_t|+|F_t|)/2}$, with A_t being the reference and F_t the rendering. The resulting SMAPE is between 0 and 2, with 0 being the reference.

6.1. Equal Time Comparison

We performed an equal time comparison between our ReSTIR FG and other similar GI resampling algorithms in Figure 6. We used the slowest algorithm (Suffix ReSTIR) as a baseline for the total render times and ensured that our ReSTIR FG remains beneath the total render times of the other algorithms.

The comparison shows that our ReSTIR FG produces results that are in line or better than the other algorithms. In all scenes, our resampling approach for final gather samples demonstrates to be better than final gathering in equal time. In an equal time comparison against ReSTIR GI, our ReSTIR FG exhibits comparable overall global illumination quality while also displaying caustics. ReSTIR PT often shows less noise than ReSTIR FG, however, fails to properly display caustic effects. Suffix ReSTIR, although the slowest technique, demonstrated more noise in all tested scenes.

The Bistro scene gets handled well by all algorithms, except of the cookie jar, where only final gathering and ReSTIR FG yield satisfactory results while the other resample algorithms fail. Among the other resampling algorithms, ReSTIR PT handles the cookie jar the best while also displaying some caustics for the wine glass. The HotelRoom scene is the most advantageous for ReSTIR FG, particularly due to the blue-colored glass surrounding the light source, which imparts a noticeable blue tint to the global illumination. ReSTIR GI and Suffix ReSTIR both fail in this regard and only show a red tint for the zoomed wall, while ReSTIR PT produces noticeably noisier results. In the Kitchen scene, ReSTIR PT shows multiple correlations, with one being enhanced in the red box (light streaks). ReSTIR GI appears visibly darker in this area, and while Suffix ReSTIR exhibits no artifacts, it is accompanied by noticeable noise. The VeachAjar scene presents no discernible issues for any of the algorithms. Our ReSTIR FG appears to be a bit more effective in finding the light paths, benefiting from light and camera tracing.

Figure 5 shows the convergence in the Veach-Ajar scene, as all algorithms are able to display this scene well. All algorithms appear to converge to the reference solution. The results are similar to Figure 6, although final gathering converges faster than Suffix ReSTIR due to the speed advantage. Similarly, ReSTIR GI passes ReSTIR PT at the start. However, ReSTIR PT seems to converge faster at the end than ReSTIR GI and ReSTIR FG. ReSTIR GI is missing the caustic effects in the shadow of the glass teapot. ReSTIR FG exhibits more noise in the corners than ReSTIR PT, which is caused by photon mapping; fewer photons are collected in corners, leading to darkening in corners.

6.2. Real Time Capabilities

As shown in Figure 6, ReSTIR PT and Suffix ReSTIR are not capable of real time rendering due to their high render times for one sample. Therefore, we conducted a comparison of the 1 SPP runtime between ReSTIR FG and ReSTIR GI, as both exhibit similar runtimes per sample. For this comparison we used scenes that

© 2024 The Authors.
Proceedings published by Eurographics - The European Association for Computer Graphics.

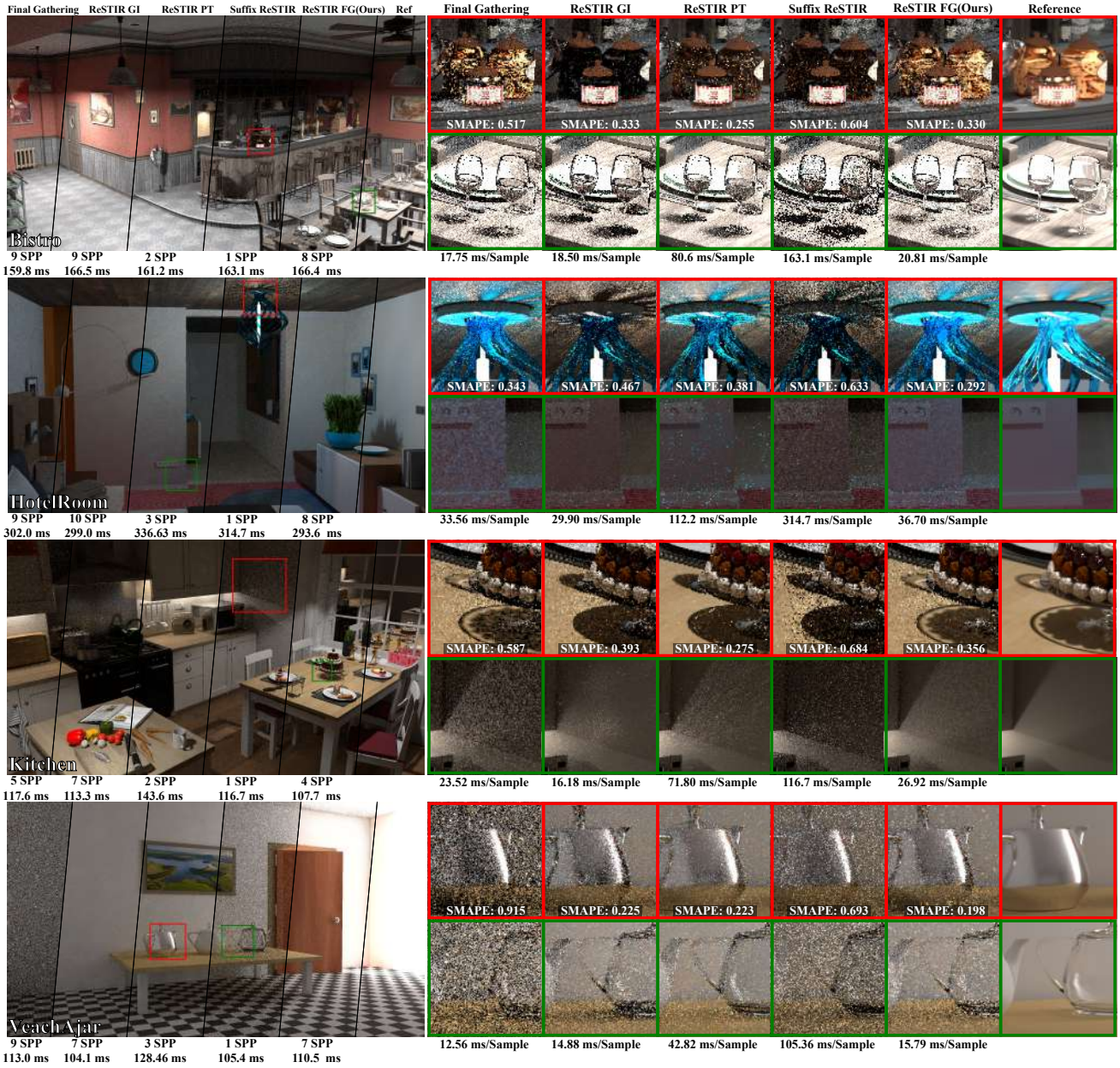


Figure 6: *Approximate equal time comparison between the other and our algorithm. To reach approximate equal time, we accumulated multiple iterations for most techniques. Render times for the G-Buffer, RTXDI and misc. are included.*

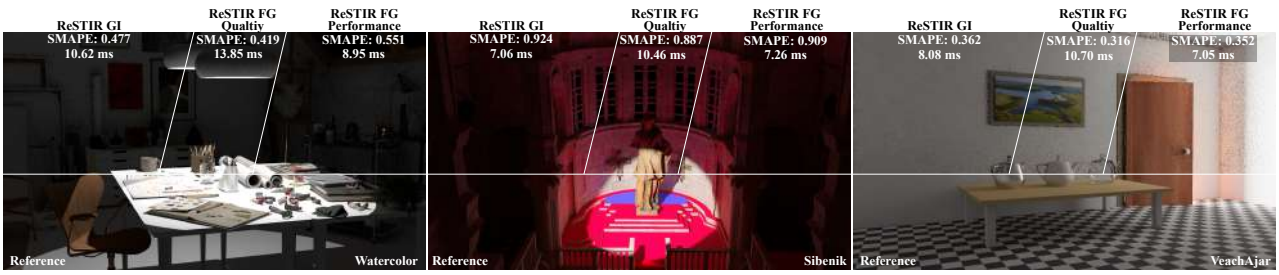


Figure 7: *1 SPP comparison between ReSTIR GI and two different quality versions of our ReSTIR FG (see Section 5).*

© 2024 The Authors.
Proceedings published by Eurographics - The European Association for Computer Graphics.

Figure 8: ***(a)*** *ReSTIR GI and ReSTIR PT are susceptible to visible correlation near specular materials, showing "blobs" (ReSTIR GI) or "streaks" (ReSTIR PT). In contrast ReSTIR FG rarely displays correlations, as the paths prone to correlations are collected directly in the form of caustics. The effect is more visible in motion (see video in Supplemental Materials).* ***(b)*** *Our ReSTIR FG is able to handle scenes with many lights ($\sim 17k$) as well as ReSTIR PT (1SPP). Scene: Old Attic from NVIDIA OMNIVERSE.*

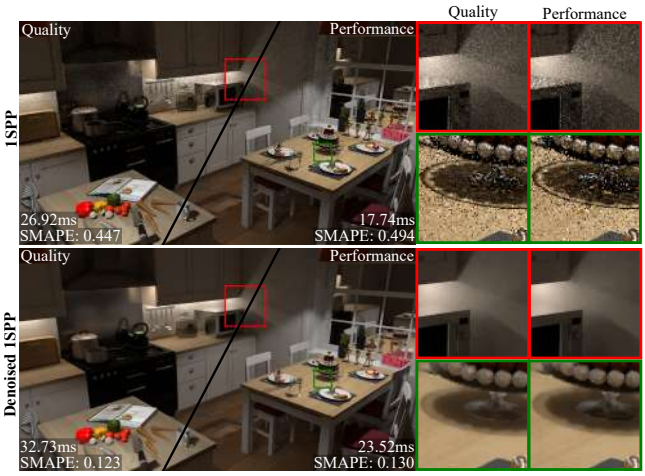


Figure 9: *Denoising comparison with two 1 SPP images of our ReSTIR FG in different quality settings (Section 5). Ray traced bias correction is used for both versions. The performance version, whether in its original or denoised form, exhibits slightly more noise compared to the quality version. However, it achieves a speedup of $\times 1.39$. The shown render times include G-Buffer, RTXDI and misc.*

do not have any significant caustic effects to mainly compare the global illumination capabilities. The comparison is illustrated in Figure 7. For ReSTIR FG we used two different quality settings, varying the number of distributed photons between them (Section 5).

Overall, the global illumination quality remains relatively similar across all three scenes. Despite being the slowest, the quality version of ReSTIR FG reliably exhibits the lowest error. ReSTIR GI often falls in the middle in terms of both runtime and error, while the performance version of ReSTIR FG achieves the best render time but also has the highest error. An exception was the Sibenik scene, where ReSTIR GI is the fastest, but also has the highest error. However, visually the noise seems indistinguishable between ReSTIR GI and both quality settings of ReSTIR FG.

Table 1 shows the render time for each part of ReSTIR FG (see Algorithm 2). Notably, decreasing the number of photons distributed in the scene directly affects the photon related passes, while the runtime of the other passes remains constant. Therefore, the numbers of photons distributed is the main quality setting for our ReSTIR FG, as less distributed photons increase performance while decreasing the quality.

6.3. Correlations

ReSTIR GI and ReSTIR PT are susceptible to correlations near specular materials, as bright, low-probability paths are resampled and propagated into neighboring pixels with spatial resampling (Figure 8a). Although these bright splats fade quickly due to the confidence cap c_i, they are very noticeable in real-time and can be seen in the supplementary video (Supplemental Materials). Our ReSTIR FG very rarely exhibits these correlations, as we directly render these cases as caustics and detect them more often due to light tracing. Suffix ReSTIR does not exhibit these correlations either, as spatial resampling is deferred to the second diffuse hit. Additionally, the final gather collection in our ReSTIR FG and in Suffix ReSTIR reduces the occurrence of these correlations.

6.4. Large Number of Lights

We also tested scenes with many emissive lights (Figure 8b). Our ReSTIR FG is able to produce results similar to ReSTIR PT, even though we approximate the global illumination using light tracing. However, we noticed that the photon generation performance was worse than in other scenes, as each photon is distributed from a random emitter. For scenes with many lights, grouping the distributed photons so they are dispatched from similar positions could improve performance.

© 2024 The Authors.
Proceedings published by Eurographics - The European Association for Computer Graphics.

Figure 10: *1 SPP rendering of San Miguel with a moving camera. The scene has multiple glass encased light sources. Top left: Other resampling algorithms either fail to locate the light source (ReSTIR GI, Suffix ReSTIR) or exhibit large correlations (ReSTIR PT). Top right: A normal caustic radius causes noise. Bottom left:A larger radius improves direct light but suffers from overblurred real caustics and splatting artifacts in less well-illuminated areas. Bottom right: Denoising handles well-illuminated areas but exhibits splats in the heavily noised regions (right wall) that only disappear if the camera remains still.*

6.5. Denoising

In Figure 9 we compared both quality version of ReSTIR FG with real time denoising using NRD's RELAX, which is an optimized version of SVFG [SKW*17]. For anti-aliasing, DLSS was used.

Similar to the comparison illustrated in Figure 7, the performance variant exhibits slightly more noise but demonstrates a faster runtime with a speedup of $\times 1.51$. However, when using denoising, the quality difference between both variants becomes marginal, while the performance increases by a factor of $\times 1.39$. Slight overall darkening and increased noise in the light reflection near the microwave can be noticed.

6.6. Disocclusion

We compared the disocclusion noise of ReSTIR FG with other resampling algorithms. Because our ReSTIR FG entirely uses screen space for resampling, it is prone to displaying more noise in disoccluded areas. However, when directly compared to ReSTIR GI and ReSTIR PT, ReSTIR FG shows less noise than ReSTIR GI and comparable or slightly less noise than ReSTIR PT in disoccluded areas. This is because photon final gathering produces better initial samples than path tracing, resulting in more useful samples that can be resampled. Suffix ReSTIR is free of disocclusion artifacts (see [KLR*23, Figure 5]), as in addition to resampling in screen space, final gather resampling is performed, by using reconnection shift on suffixes created by nearby prefixes. A direct comparison of the disocclusion is shown in the supplemental video (Supplemental Materials).

6.7. Glass Encased Light Sources

Realistic light sources are often encased by glass. As we perform both light and camera path tracing, rendering scenes with such light sources is possible. The other resampling algorithms have problems with those scenes since they are all based on camera path tracing and have to randomly hit the light source, as directly connecting to the light source does not work. This results in visible correlations for ReSTIR PT, while ReSTIR GI and Suffix ReSTIR fail to find the light paths. Figure 10 shows renderings of the San Miguel scene with glass encased lamps as light sources. However, while our ReSTIR FG can handle those scenes, it some caveats shown in Figure 10 that would need to be improved on.

6.8. Limitations

The performance of ReSTIR FG is highly dependent on the light distribution throughout the scene. When many photons overlap, performance can drop drastically due to the linear photon collection process. Additionally, performance can change significantly with variations in photon radii, as a large radius causes photon overlapping, while very small photon radii can lead to correlations during resampling. This necessitates to per-scene fine-tuning of the photon radius.

Depending on the materials used in a scene, caustic photons can lead to visible splats, akin to fireflies, all over the scene. This occurrence is more prevalent when a well-illuminated metal surface scatters photons across the scene. To mitigate these artifacts, adjustments to the caustic photon definition may be necessary. Various strategies can be employed, such as restricting caustics to direct light only (avoiding the storage of photons after the first diffuse bounce), fine-tuning material properties, or stochastically classifying caustic photons as global photons depending on the distance traveled. However, no single solution seems universally applicable across all situations. Scenes with glass encased light sources (Section 6.7, Figure 10) are also a special case of this problem, where additional caustic definition would have to be defined to get big photons for direct light and small photons for real caustics.

7. Discussion

We presented a novel algorithm, ReSTIR FG, that combines ReSTIR with ray tracing hardware optimized photon mapping to achieve competitive global illumination simulation capable of rendering caustics in real time. To this end, we introduced a GPU-optimized version of photon final gathering that quickly generates consistent images with global illumination and caustics. The photon final gather samples and caustic photons are resampled using ReSTIR to achieve real-time global illumination.

Compared to other state-of-the-art global illumination algorithms, our ReSTIR FG stands out as the only method that consistently renders all caustic effects while maintaining competitive render times and image quality. We demonstrated that ReSTIR FG delivers global illumination with quality and performance similar to ReSTIR GI, while showing no correlation in scenes with specular materials. Additionally, ReSTIR FG can perform better than ReSTIR GI by distributing fewer photons, resulting in a slightly

© 2024 The Authors.
Proceedings published by Eurographics - The European Association for Computer Graphics.

noisier image that shows no noticeable difference when denoising is applied. While ReSTIR PT generally produces less noisy images per iteration than ReSTIR FG, its render times are not suitable for real-time applications. ReSTIR PT handles metals and glass better than ReSTIR FG, while our algorithm can render caustics better and shows no correlations near specular materials. Suffix ReSTIR, the slowest and noisiest algorithm but exhibits no correlations and no disocclusion artifacts. However, Suffix ReSTIR is inefficient at rendering caustics.

8. Conclusion and Future Work

With ReSTIR FG, we presented a new addition to the ReSTIR algorithms by combining ReSTIR with light tracing. Our algorithm is competitive with state-of-the-art resampling algorithms while standing out as the sole technique that can effectively render caustics.

As shown, the global illumination for ReSTIR FG works well, even if only a small number of photons are distributed. To combat the resulting splatting artifacts caused by the caustics, techniques similar to photon guiding [GPGSK18] could be used to increase the photon quality while reducing the needed number of photon paths.

As the noise from photon mapping and a path tracer is different, a special denoiser developed for photons could be used for the caustics. This could improve the temporal lag while also avoiding the overblurring from NRD. A special denoiser could be especially useful for scenes with glass encased light sources (Figure 10). Initial endeavors in developing a denoiser for photon mapping were done using machine learning [ZWW*20]. Unfortunately, the proposed denoiser exhibits limitations, particulary its inability to handle extreme noise and camera movement, which makes it unsuitable for ReSTIR FG.

As ReSTIR FG exhibits more noise on metal and glass surfaces, it could benefit from a using hybrid shift [LKB*22] for the first diffuse hit, similar to Suffix ReSTIR. The path to the first diffuse hit could then be resampled with temporal GRIS. Additionally, it would be interesting to see how our algorithm performs if the final gather resampling step is deferred to the second diffuse surface hit, as done in Suffix ReSTIR.

Supplemental Materials

Full source code for our implementations and videos are available at: https://github.com/TU-Clausthal-Rendering/ReSTIR-FG

Acknowledgements

We would like to thank the reviewers for their valuable feedback and especially the reviewer who proposed to collect caustic photons as a ReSTIR process.

We also like to thank the creators of the various scenes used in this work: The Amazon Lumberyard Bistro [Ama17], used in Figures 1, 4 and 6. The Kitchen scene by Jay-Artist [JA12], which we modified and used in Figures 3, 6, 8 and 9. The Watercolor scene by Angelo Ferretti from prbt-4, used in Figure 7. The Sibenik Cathedral by Marko Dabrovic, modified by us and used in Figure 7. The Old Attic scene from NVIDIA OMNIVERSE (Figure 8). The San Miguel scene [LMC*17] by Guillermo M. Leal Llaguno used in Figure 10.

References

[Ama17] AMAZON LUMBERYARD: Amazon Lumberyard Bistro, Open Research Content Archive (ORCA), 2017. URL: http://developer.nvidia.com/orca/amazon-lumberyard-bistro. 12

[BJW21] BOKSANSKY J., JUKARAINEN P., WYMAN C.: *Rendering Many Lights with Grid-Based Reservoirs*. Apress, Berkeley, CA, 2021, pp. 351–365. doi:10.1007/978-1-4842-7185-8_23. 2

[Boi21] BOISSÉ G.: World-space spatiotemporal reservoir reuse for ray-traced global illumination. In *SIGGRAPH Asia 2021 Technical Communications* (New York, NY, USA, 2021), SA '21 Technical Communications, Association for Computing Machinery. URL: https://doi.org/10.1145/3478512.3488613, doi:10.1145/3478512.3488613. 2

[BWP*20] BITTERLI B., WYMAN C., PHARR M., SHIRLEY P., LEFOHN A., JAROSZ W.: Spatiotemporal reservoir resampling for real-time ray tracing with dynamic direct lighting. *ACM Trans. Graph. 39*, 4 (2020), 148–1. doi:10.1145/3386569.3392481. 1, 2, 4, 7

[Cha82] CHAO M.-T.: A general purpose unequal probability sampling plan. *Biometrika 69*, 3 (1982), 653–656. doi:10.2307/2336002. 1, 2, 4

[DKHS14] DAVIDOVIČ T., KŘIVÁNEK J., HAŠAN M., SLUSALLEK P.: Progressive light transport simulation on the GPU: Survey and improvements. *ACM Trans. Graph. 33*, 3 (2014). doi:10.1145/2602144. 2

[DS05] DACHSBACHER C., STAMMINGER M.: Reflective shadow maps. In *Proceedings of the 2005 Symposium on Interactive 3D Graphics and Games* (New York, NY, USA, 2005), I3D '05, Association for Computing Machinery, p. 203–231. doi:10.1145/1053427.1053460. 2

[EPVV21] EVANGELOU I., PAPAIOANNOU G., VARDIS K., VASILAKIS A. A.: Fast radius search exploiting ray tracing frameworks. *Journal of Computer Graphics Techniques (JCGT) 10*, 1 (2021), 25–48. URL: http://jcgt.org/published/0010/01/02/. 2, 3, 5

[GKDS12] GEORGIEV I., KŘIVÁNEK J., DAVIDOVIČ T., SLUSALLEK P.: Light transport simulation with vertex connection and merging. *ACM Trans. Graph. 31*, 6 (2012). doi:10.1145/2366145.2366211. 2

[GPGSK18] GRITTMANN P., PÉRARD-GAYOT A., SLUSALLEK P., KŘIVÁNEK J.: Efficient caustic rendering with lightweight photon mapping. *Computer Graphics Forum 37*, 4 (2018). EGSR '18. doi:10.1111/cgf.13481. 12

[HJ09] HACHISUKA T., JENSEN H. W.: Stochastic progressive photon mapping. In *ACM SIGGRAPH Asia 2009 Papers* (New York, NY, USA, 2009), SIGGRAPH Asia '09, Association for Computing Machinery. doi:10.1145/1661412.1618487. 2

[HJ10] HACHISUKA T., JENSEN H. W.: Parallel progressive photon mapping on GPUs. In *ACM SIGGRAPH ASIA 2010 Sketches* (New York, NY, USA, 2010), SA '10, Association for Computing Machinery. doi:10.1145/1899950.1900004. 2

[HOJ08] HACHISUKA T., OGAKI S., JENSEN H. W.: Progressive photon mapping. In *ACM SIGGRAPH Asia 2008 Papers* (New York, NY, USA, 2008), SIGGRAPH Asia '08, Association for Computing Machinery. doi:10.1145/1457515.1409083. 2

[JA12] JAY-ARTIST: Country-kitchen cycles, 2012. URL: https://blendswap.com/blend/5156. 12

[Jen96] JENSEN H. W.: Global illumination using photon maps. In *Rendering Techniques' 96: Proceedings of the Eurographics Workshop*

© 2024 The Authors.
Proceedings published by Eurographics - The European Association for Computer Graphics.

in Porto, Portugal, June 17–19, 1996 7 (1996), Springer, pp. 21–30. doi:10.1007/978-3-7091-7484-5_3. 1, 2, 3

[Kaj86] KAJIYA J. T.: The rendering equation. In *Proceedings of the 13th Annual Conference on Computer Graphics and Interactive Techniques* (New York, NY, USA, 1986), SIGGRAPH '86, Association for Computing Machinery, p. 143–150. doi:10.1145/15922.15902. 1, 2, 3

[KBG23] KERN R., BRÜLL F., GROSCH T.: Accelerated photon mapping for hardware-based ray tracing. *Journal of Computer Graphics Techniques (JCGT) 12*, 1 (2023), 1–28. URL: http://jcgt.org/published/0012/01/01/. 2, 3, 5, 7

[KCK*22] KALLWEIT S., CLARBERG P., KOLB C., DAVIDOVIČ T., YAO K.-H., FOLEY T., HE Y., WU L., CHEN L., AKENINE-MÖLLER T., WYMAN C., CRASSIN C., BENTY N.: The Falcor rendering framework, 2022. URL: https://github.com/NVIDIAGameWorks/Falcor. 7

[KD13] KAPLANYAN A. S., DACHSBACHER C.: Adaptive progressive photon mapping. *ACM Trans. Graph. 32*, 2 (2013). doi:10.1145/2451236.2451242. 2

[Kim19] KIM H.: Caustics using screen-space photon mapping. In *Ray Tracing Gems*. Springer, 2019, pp. 543–555. doi:10.1007/978-1-4842-4427-2_30. 2

[KLR*23] KETTUNEN* M., LIN* D., RAMAMOORTHI R., BASHFORD-ROGERS T., WYMAN C.: Conditional resampled importance sampling and ReSTIR. In *SIGGRAPH Asia (Conference Track)* (2023). (*Joint First Authors). doi:10.1145/3610548.3618245. 1, 2, 7, 11

[KMA*15] KETTUNEN M., MANZI M., AITTALA M., LEHTINEN J., DURAND F., ZWICKER M.: Gradient-domain path tracing. *ACM Trans. Graph. 34*, 4 (2015). doi:10.1145/2766997. 6

[KZ11] KNAUS C., ZWICKER M.: Progressive photon mapping: A probabilistic approach. *ACM Trans. Graph. 30*, 3 (2011). doi:10.1145/1966394.1966404. 2, 8

[LKB*22] LIN* D., KETTUNEN* M., BITTERLI B., PANTALEONI J., YUKSEL C., WYMAN C.: Generalized resampled importance sampling: Foundations of ReSTIR. *ACM Trans. Graph. (Proceedings of SIGGRAPH 2022) 41*, 4 (2022), 75:1–75:23. (*Joint First Authors). doi:10.1145/3528223.3530158. 1, 2, 3, 4, 7, 12

[LKL*13] LEHTINEN J., KARRAS T., LAINE S., AITTALA M., DURAND F., AILA T.: Gradient-domain metropolis light transport. *ACM Trans. Graph. 32*, 4 (2013). URL: https://doi.org/10.1145/2461912.2461943, doi:10.1145/2461912.2461943. 5, 6

[LMC*17] LLAGUNO G. M. L., MCGUIRE M., CARDENAS G., MARA M., HULL N.: San Miguel 2.0, 2017. URL: https://casual-effects.com/data. 12

[MGNM19] MAJERCIK Z., GUERTIN J.-P., NOWROUZEZAHRAI D., MCGUIRE M.: Dynamic diffuse global illumination with ray-traced irradiance fields. *Journal of Computer Graphics Techniques (JCGT) 8*, 2 (2019), 1–30. URL: http://jcgt.org/published/0008/02/01/. 2

[MLM13] MARA M., LUEBKE D., MCGUIRE M.: Toward practical real-time photon mapping: Efficient GPU density estimation. In *Proceedings of the ACM SIGGRAPH Symposium on Interactive 3D Graphics and Games* (New York, NY, USA, 2013), I3D '13, Association for Computing Machinery, p. 71–78. doi:10.1145/2448196.2448207. 2

[MMK*22] MAJERCIK Z., MÜLLER T., KELLER A., NOWROUZEZAHRAI D., MCGUIRE M.: Dynamic diffuse global illumination resampling. *Computer Graphics Forum 41*, 1 (2022), 158–171. doi:https://doi.org/10.1111/cgf.14427. 2

[MMSM21] MAJERCIK Z., MARRS A., SPJUT J., MCGUIRE M.: Scaling probe-based real-time dynamic global illumination for production. *Journal of Computer Graphics Techniques (JCGT) 10*, 2 (2021), 1–29. URL: http://jcgt.org/published/0010/02/01/. 2

[OLK*21] OUYANG Y., LIU S., KETTUNEN M., PHARR M., PANTALEONI J.: ReSTIR GI: Path resampling for real-time path tracing. *Computer Graphics Forum 40*, 8 (2021), 17–29. doi:10.1111/cgf.14378. 1, 2, 7

[SA19] SMAL N., AIZENSHTEIN M.: Real-time global illumination with photon mapping. In *Ray Tracing Gems*. Apress, Berkeley, CA, 2019, pp. 409–436. doi:10.1007/978-1-4842-4427-2_24. 2

[SKW*17] SCHIED C., KAPLANYAN A., WYMAN C., PATNEY A., CHAITANYA C. R. A., BURGESS J., LIU S., DACHSBACHER C., LEFOHN A., SALVI M.: Spatiotemporal variance-guided filtering: Real-time reconstruction for path-traced global illumination. In *Proceedings of High Performance Graphics* (New York, NY, USA, 2017), HPG '17, Association for Computing Machinery. doi:10.1145/3105762.3105770. 1, 11

[SLK*24] SAWHNEY R., LIN D., KETTUNEN M., BITTERLI B., RAMAMOORTHI R., WYMAN C., PHARR M.: Decorrelating restir samplers via mcmc mutations. *ACM Trans. Graph. 43*, 1 (2024). doi:10.1145/3629166. 2

[TCE05] TALBOT J., CLINE D., EGBERT P.: Importance Resampling for Global Illumination. In *Eurographics Symposium on Rendering (2005)* (2005), Bala K., Dutre P., (Eds.), The Eurographics Association. doi:10.2312/EGWR/EGSR05/139-146. 1, 2, 3

[Tok23] TOKUYOSHI Y.: Efficient spatial resampling using the pdf similarity. *Proc. ACM Comput. Graph. Interact. Tech. 6*, 1 (2023). doi:10.1145/3585501. 2

[VG95] VEACH E., GUIBAS L. J.: Optimally combining sampling techniques for monte carlo rendering. In *Proceedings of the 22nd Annual Conference on Computer Graphics and Interactive Techniques* (New York, NY, USA, 1995), SIGGRAPH '95, Association for Computing Machinery, p. 419–428. doi:10.1145/218380.218498. 4

[VG97] VEACH E., GUIBAS L. J.: Metropolis light transport. In *Proceedings of the 24th Annual Conference on Computer Graphics and Interactive Techniques* (USA, 1997), SIGGRAPH '97, ACM Press/Addison-Wesley Publishing Co., p. 65–76. doi:10.1145/258734.258775. 1

[Vit85] VITTER J. S.: Random sampling with a reservoir. *ACM Trans. Math. Softw. 11*, 1 (1985), 37–57. doi:10.1145/3147.3165. 1, 2, 3, 4, 5

[WKL*23] WYMAN C., KETTUNEN M., LIN D., BITTERLI B., YUKSEL C., JAROSZ W., KOZLOWSKI P.: A gentle introduction to restir path reuse in real-time. In *ACM SIGGRAPH 2023 Courses* (New York, NY, USA, 2023), SIGGRAPH '23, Association for Computing Machinery. URL: https://doi.org/10.1145/3587423.3595511, doi:10.1145/3587423.3595511. 2, 3, 4

[WP21] WYMAN C., PANTELEEV A.: Rearchitecting Spatiotemporal Resampling for Production. In *High-Performance Graphics - Symposium Papers* (2021), Binder N., Ritschel T., (Eds.), The Eurographics Association. doi:10.2312/hpg.20211281. 2, 4, 6, 7

[YO21] YANG X., OUYANG Y.: Real-time ray traced caustics. In *Ray Tracing Gems II*. Springer, 2021, pp. 469–497. doi:10.1007/978-1-4842-7185-8_30. 2

[ZWW*20] ZENG Z., WANG L., WANG B.-B., KANG C.-M., XU Y.-N.: Denoising stochastic progressive photon mapping renderings using a multi-residual network. *Journal of Computer Science and Technology 35* (2020), 506–521. doi:10.1007/s11390-020-0264-1. 12

© 2024 The Authors.
Proceedings published by Eurographics - The European Association for Computer Graphics.